

Team: Van Duong, Stephen Shao, Andy Neidhart

Course: CMPE 138

Professor: Dr. Gheorghe Guzun

Name Trends Analysis in the United States Using BigQuery Report

Introduction/Motivation/Problem Definition

For this project, we analyzed the `bigquery-public-data.usa_names.usa_1910_current` dataset, which contains U.S. baby name records such as, birth state, gender, year, name, and count fields from the years 1910 all the way to current day. This is good for applications involving demographic analysis, cultural trend tracking, or even something like a baby name recommendation tool. This dataset is particularly interesting because it can show us shifts in culture, gender norms, and regional diversity in a way that would otherwise be hard to see.

Potential users of such applications could be parents seeking name inspiration, researchers studying naming trends, or companies interested in generational marketing. The data could be used in interactive dashboards, trend reports, or embedded into apps that visualize name popularity by year, state, or gender.

This dataset has shown to be challenging due to its size, there are millions of records and the need for efficient aggregation and filtering is key. Manual analysis would definitely not be feasible, so we relied on SQL queries in BigQuery to extract different trends, such as the most popular names by decade, shifts in gender-neutral names, or the impact of celebrities on naming spikes. In this project we primarily focus on query optimization, using techniques like filtering by decade and grouping by relevant fields to reduce scan costs and improve performance. We chose these algorithms because they are reproducible and scalable for large, structured datasets.

Related Work

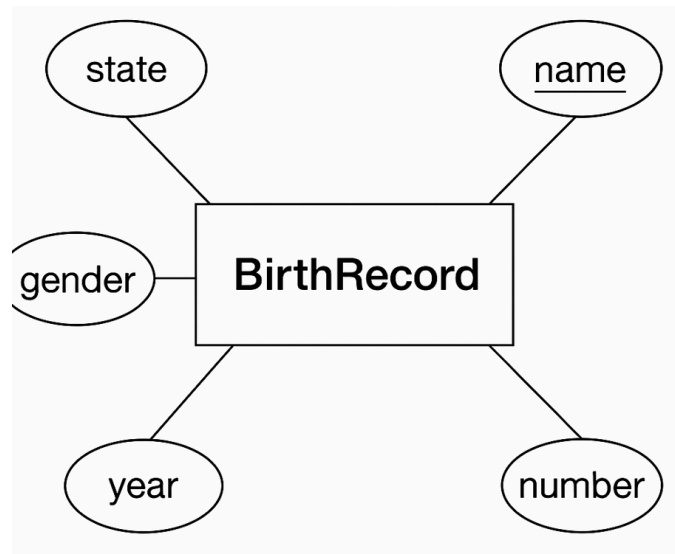
Some related works that are similar to the `usa_1910_current` dataset would be the US social security baby names dataset. This would contain a dataset ever since the 1880 of information of each newborn baby. Similar to `usa_1910_current` dataset, it would contain names, gender, year. But in this dataset, it would be more time-based. The amount of data size would be a lot bigger compared to the USSA dataset. The citation for the USSA dataset is written "Popular Baby Names." *Social Security Administration*, www.ssa.gov/oact/babynames/. Accessed 1 May 2025." For the project, there are some pros and cons towards developing this project. Pros would include the amount of valuable data in `usa_1910_current` dataset. The amount of data that was given ever since 1910 shows us the evolution of names and the trend in which people would name their kids in spans of years. A con would be in this dataset, the changes in names would be shown, but not the cause of why it might have happened. The cause and effect for this dataset was not shown and could not be detected even though it shows a trend of names in the dataset.

For our project, there are features that were made including most popular names by decade, gender-neutral names, states with most unique names, fastest rising names after 1990, celebrity name spike, and stable names across decades.

SQL queries are needed for each feature

Team Member	Feature	SQL Query Summary
Van	Most Popular Name by Decade	GROUP BY name, decade + SUM(number) + ROW_NUMBER() to get top name per decade
Van	Gender-Neutral Names	Use SUM(CASE WHEN gender='M' THEN number) and compare with F, filter with HAVING ABS(diff) < 1000
Stephen	States with Most Unique Names	GROUP BY state + COUNT(DISTINCT name) to find name diversity
Stephen	Fastest Rising Names After 1990	Join two decade ranges, compute percent increase between 1990s and 2000s
Andy	Celebrity Influence – 'Kobe'	WHERE name = 'Kobe' + GROUP BY year to visualize name usage over time
Andy	Stable Names Across Decades	GROUP BY name, decade + STDDEV() and COUNT() to find consistent names

The ER Diagram of our database



Component	Type	Description
BirthRecord	Entity	Main entity representing each record of baby name usage.
state	Attribute	U.S. state abbreviation (e.g., CA, NY) where the name record was reported.
gender	Attribute	'M' or 'F' indicating the baby's gender.
year	Attribute	Year of birth (e.g., 1910, 2020).
name	Key Attribute (underlined)	Baby's first name. Combined with other fields, forms a composite key.
number	Attribute	Total number of babies given that name in the specific year, state, gender.

Link to Github repository with SQL files

Here is our github link:

<https://github.com/vduong421/name-trend-analysis.git>

Analysis of Queries

Most important queries and Features responsible for

Two most important queries we worked on were “Most popular name by decade” and “gender-neutral names”. The most popular name by decade highlights the top baby name in each decade and it uses the combination of GROUP BY and ROW_NUMBER() window function. The gender-neutral names detects names uses conditional aggregation CASE WHEN and HAVING filter in order to detect names with near-equal count for male and female. These queries are important to show long-term cultural shifts in naming and analyze the naming trends.

Inputs/outputs

Input:

All queries use “bigquery-public-data.us_names.usa_1910_current” table from google cloud with these fields: name, gender, year, state, number

Output:

Van's Queries

Query 1: Most Popular Name by Decade

Query 2: Gender-Neutral Names

Stephen's Queries

Query 1: States with Most Unique Names

Query 2: Fastest Rising Names After 1990

Andy's Queries

Query 1: Celebrity Influence — 'Kobe'

Query 2: Stable Names Across Decades

Optimization possibilities (indexes, etc.)

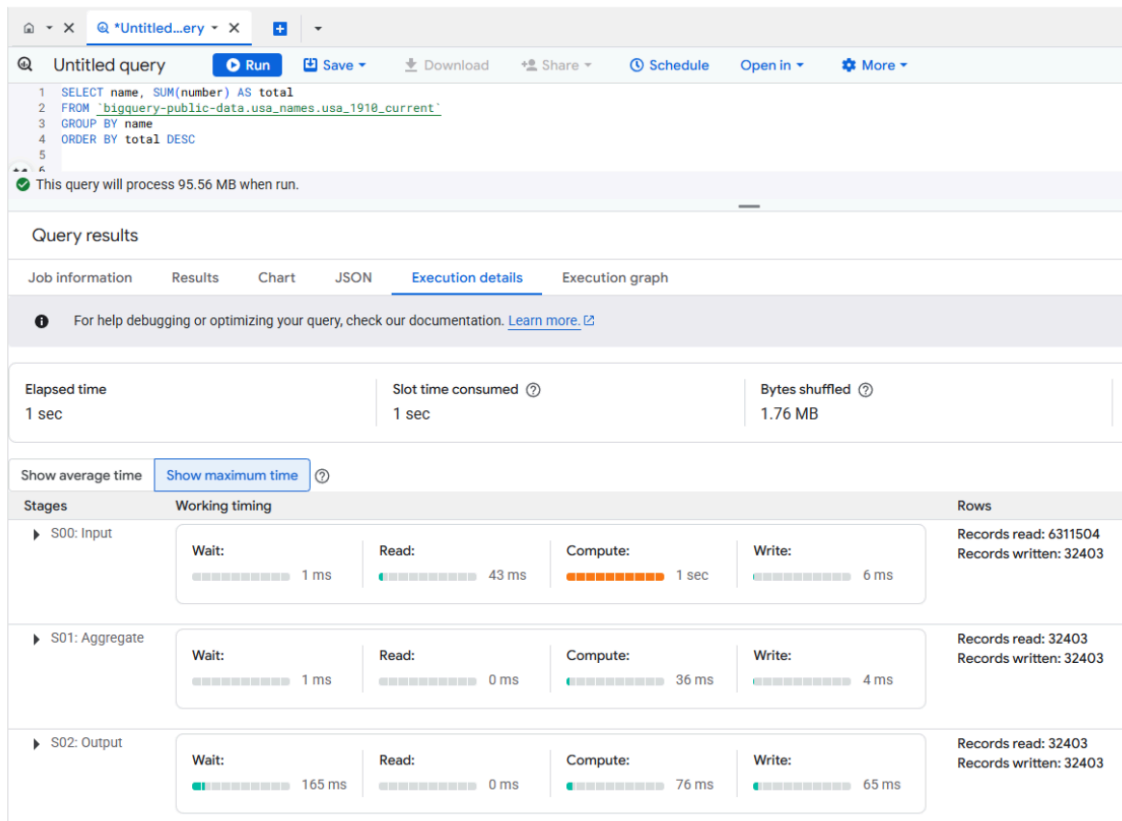
Bigquery doesn't use traditional indexes so we have to apply these optimizations in order to improve IO cost:

- We use "WHERE year BETWEEN" in order to filter row and reduce data scanned
- We avoid "SELECT *" in order to avoid select all rows. We only select needed data
- We also use "GROUP BY" in order to reduce intermediate result size

IO cost estimates with/without optimization

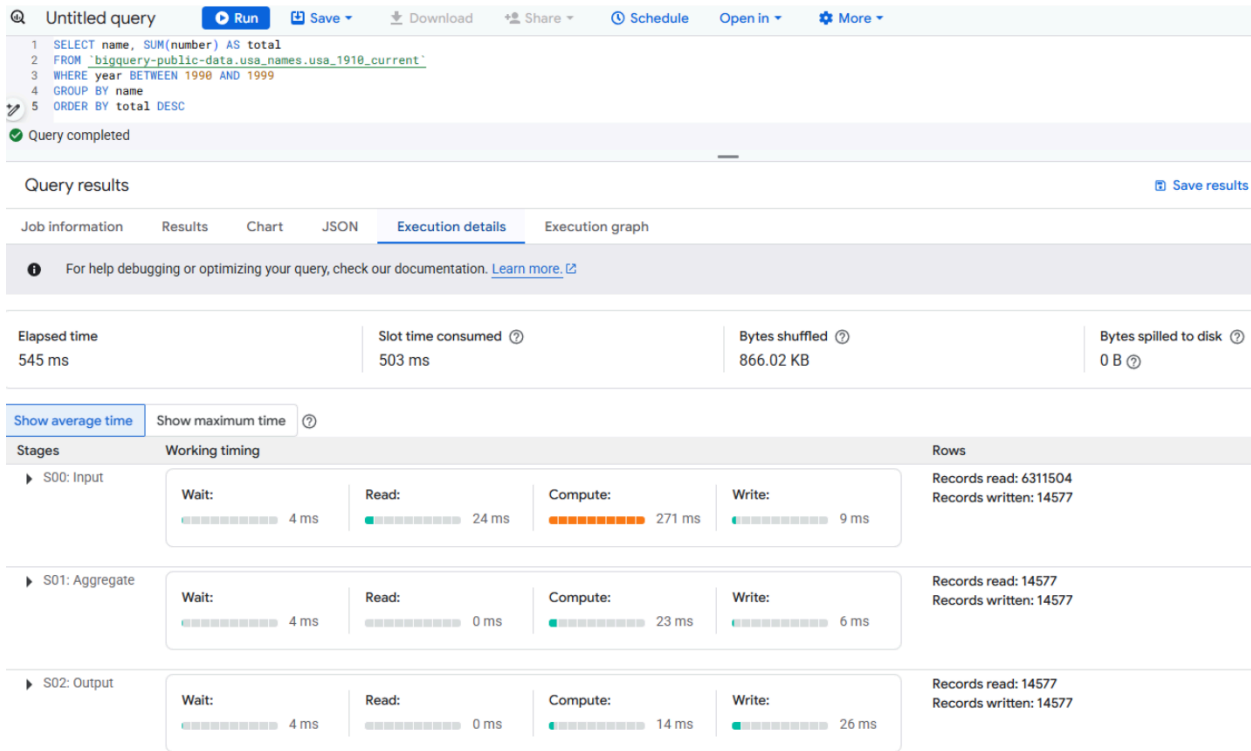
Here is the IO cost without optimization:

Query Optimization (Unoptimized)



Here is the IO cost with optimization:

Query Optimization (Optimized)



From that, we can have a comparison result

Query Optimization

Result?

Metric	Unoptimized Query	Optimized Query (Filtered by decade)
Elapsed Time	1 sec	545 ms
Slot Time Consumed	1 sec	503 ms
Bytes Shuffled	1.76 MB	866 KB
Records Read	6,311,504	6,311,504 (same)
Records Written	32,403	14,577
Compute Time (Stage 0)	1 sec	271 ms

As you can see in the figure above, the query took 1 second and shuffled 1.76 MB of data while the optimized version ran 545 milliseconds and shuffled only 866 kb. This is a huge save since it writes fewer rows, drop records written from 32.403 to 14.577 and stage 0 compute time was reduced by over 70%. This shows a huge improvement from a small SQL change

Results and Findings

Measure running time (before and after optimization)

As I explained above, the unoptimized version ran in 1 second while the optimized one only ran in 545 ms. This improves nearly 2x speedup

Graphs/tables with results

Van's Queries

Query 1: Most Popular Name by Decade

Van's Queries

✓

▶

```
# Query 1: Most popular name by decade
query_van_1 = """
WITH Name_Decade AS (
  SELECT
    name,
    SUM(number) AS total_births,
    CAST(year/10 AS INT64)*10 AS decade # Changed to CAST to INT64
  FROM `bigquery-public-data.usa_names.usa_1910_current`
  GROUP BY name, decade
)
SELECT decade, name, total_births
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY decade ORDER BY total_births DESC) as rn
  FROM Name_Decade
)
WHERE rn = 1
ORDER BY decade
"""

df_van_1 = run_query(query_van_1)
print("Van Query 1: Most Popular Name by Decade")
print(df_van_1)
```

↗

Van Query 1: Most Popular Name by Decade

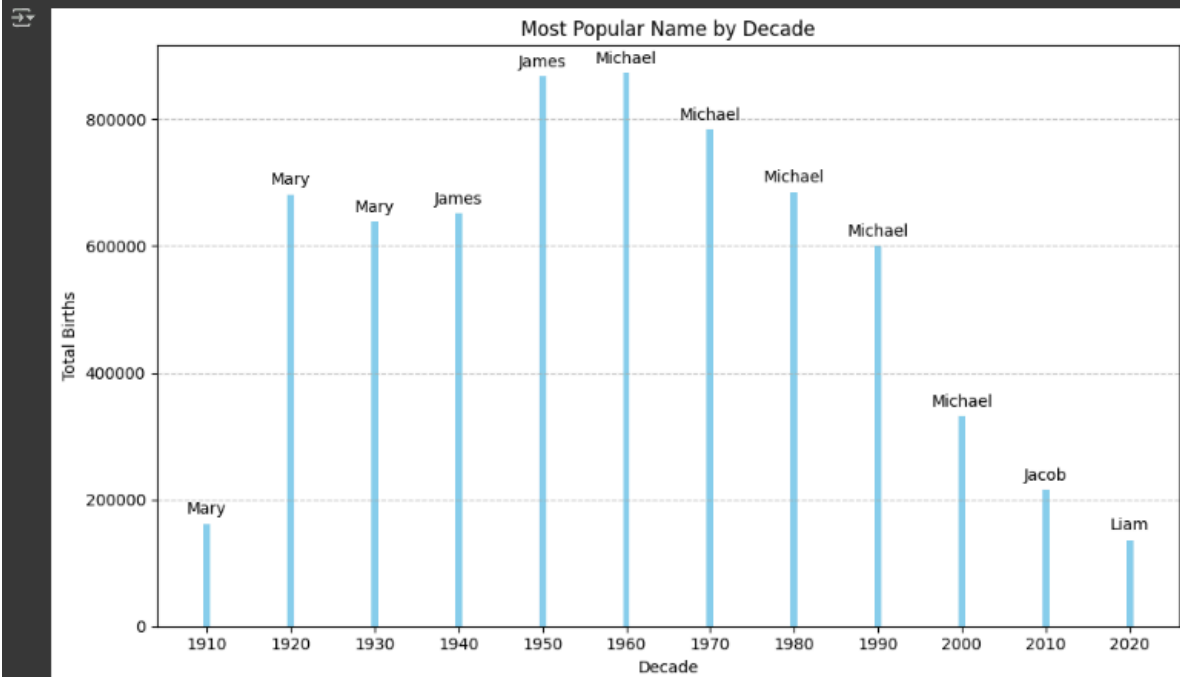
	decade	name	total_births
0	1910	Mary	161892
1	1920	Mary	680783
2	1930	Mary	638887
3	1940	James	650818
4	1950	James	867126
5	1960	Michael	872620
6	1970	Michael	784211
7	1980	Michael	684431
8	1990	Michael	600070
9	2000	Michael	331135
10	2010	Jacob	215500
11	2020	Liam	136044

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.bar(df_van_1['decade'], df_van_1['total_births'], color='skyblue')
plt.xlabel('Decade')
plt.ylabel('Total Births')
plt.title('Most Popular Name by Decade')
plt.xticks(df_van_1['decade'])
plt.grid(axis='y', linestyle='--', alpha=0.7)

for i, name in enumerate(df_van_1['name']):
    plt.text(df_van_1['decade'][i], df_van_1['total_births'][i] + 10000, name, ha='center', va='bottom')

plt.tight_layout()
plt.show()
```



Query 2: Gender-Neutral Names

```
# Query 2: Gender-neutral names
query_van_2 = """
WITH Name_Gender AS (
  SELECT name, gender, SUM(number) AS total_births
  FROM `bigquery-public-data.usa_names.usa_1910_current`
  GROUP BY name, gender
),
Name_Totals AS (
  SELECT name,
    SUM(CASE WHEN gender='M' THEN total_births ELSE 0 END) AS male_count,
    SUM(CASE WHEN gender='F' THEN total_births ELSE 0 END) AS female_count,
    SUM(total_births) AS total_count -- Calculate total births for each name
  FROM Name_Gender
  GROUP BY name
  HAVING ABS(SUM(CASE WHEN gender='M' THEN total_births ELSE 0 END) - SUM(CASE WHEN gender='F' THEN total_births ELSE 0 END)) < 1000
)
SELECT name, male_count, female_count
FROM Name_Totals
ORDER BY total_count DESC -- Sort by total births in descending order
LIMIT 20 -- Limit to the top 20 names
"""

df_van_2 = run_query(query_van_2)
print("Van Query 2: Gender Neutral Names (Top 20)")
print(df_van_2)
```

Van Query 2: Gender Neutral Names (Top 20)

	name	male_count	female_count
0	Kerry	46363	45622
1	Justice	16796	16561
2	Kris	10979	11020
3	Stevie	9500	9711
4	Unknown	7208	7501
5	Baby	4664	4166
6	Infant	3971	3684
7	Arden	3563	3784
8	Landry	3569	3767
9	Palmer	3375	3680
10	Briar	3243	3157
11	Yael	3050	2549
12	Austyn	3095	2425
13	Jael	1875	2193
14	Maxie	2135	1827
15	Camdyn	2350	1356
16	Lorenza	1900	1466
17	Claudie	1478	1882
18	Ryley	1849	1422
19	Ocie	1087	1938

```

import matplotlib.pyplot as plt
import numpy as np

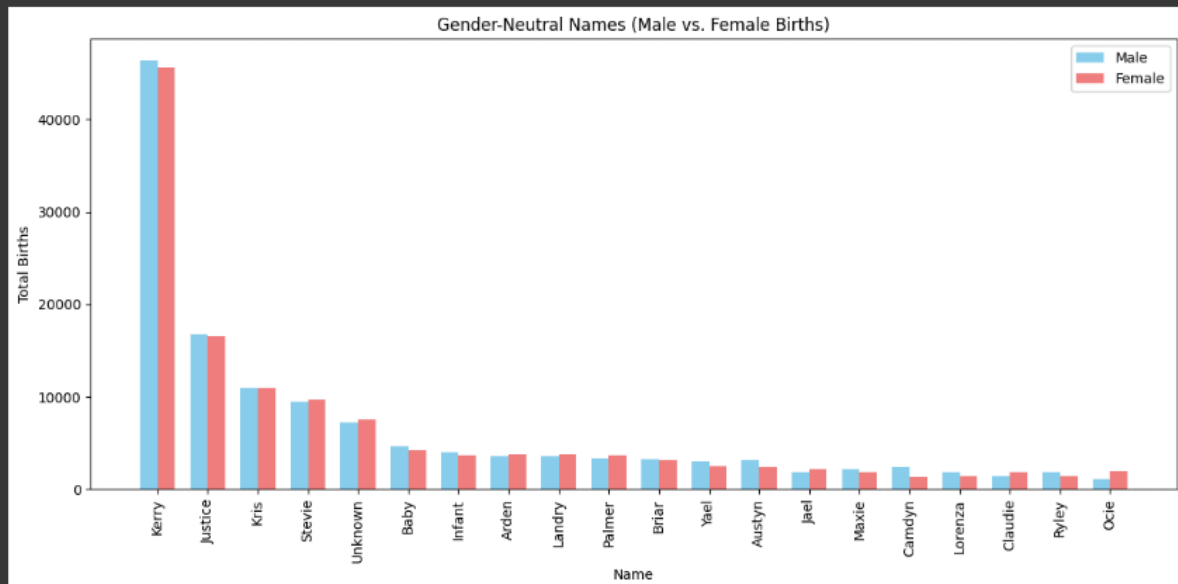
names = df_van_2['name']
male_counts = df_van_2['male_count']
female_counts = df_van_2['female_count']

bar_width = 0.35
x_pos = np.arange(len(names))

plt.figure(figsize=(12, 6))
plt.bar(x_pos, male_counts, bar_width, label='Male', color='skyblue')
plt.bar(x_pos + bar_width, female_counts, bar_width, label='Female', color='lightcoral')

plt.xticks(x_pos + bar_width / 2, names, rotation=90)
plt.xlabel('Name')
plt.ylabel('Total Births')
plt.title('Gender-Neutral Names (Male vs. Female Births)')
plt.legend()
plt.tight_layout()
plt.show()

```



Stephen's Queries

Query 1: States with Most Unique Names

▼ Stephen's Queries

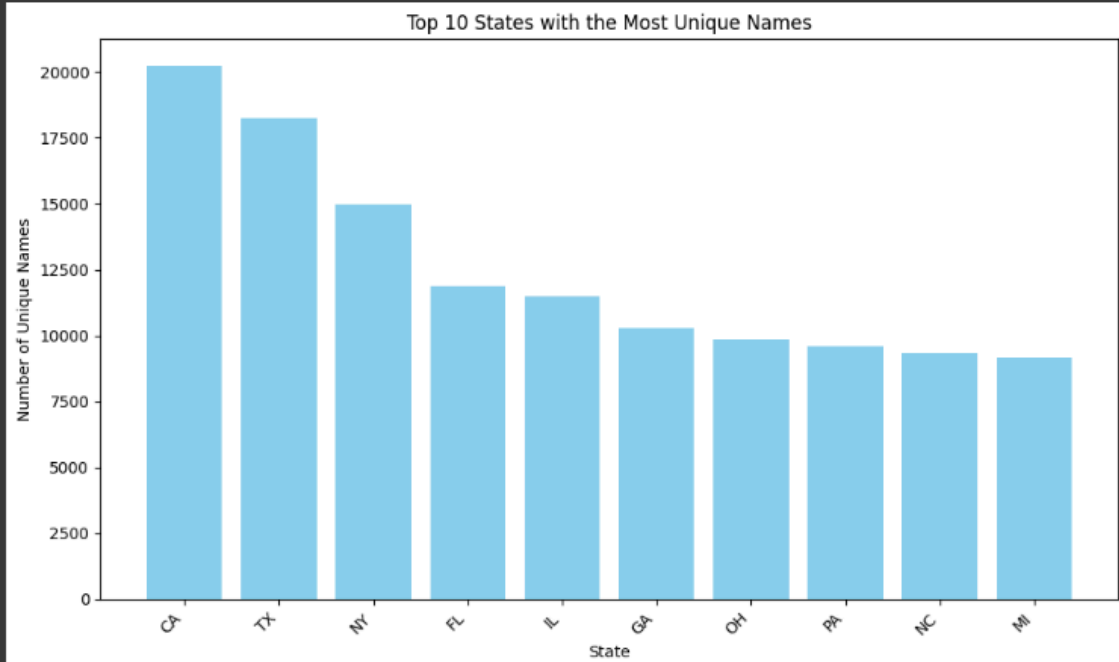
```
[11]
# Query 1: States with most unique names
query_stephen_1 = """
SELECT state, COUNT(DISTINCT name) AS unique_names
FROM `bigquery-public-data.usa_names.usa_1910_current`
GROUP BY state
ORDER BY unique_names DESC
LIMIT 10
"""

df_stephen_1 = run_query(query_stephen_1)
print("Stephen Query 1: States with Most Unique Names")
print(df_stephen_1)
```

```
↗ Stephen Query 1: States with Most Unique Names
  state  unique_names
0    CA           20239
1    TX           18238
2    NY           14997
3    FL           11869
4    IL           11493
5    GA           10284
6    OH            9844
7    PA            9582
8    NC            9345
9    MI            9178
```

```
[12] import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.bar(df_stephen_1['state'], df_stephen_1['unique_names'], color='skyblue')
plt.xlabel('State')
plt.ylabel('Number of Unique Names')
plt.title('Top 10 States with the Most Unique Names')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Query 2: Fastest Rising Names After 1990

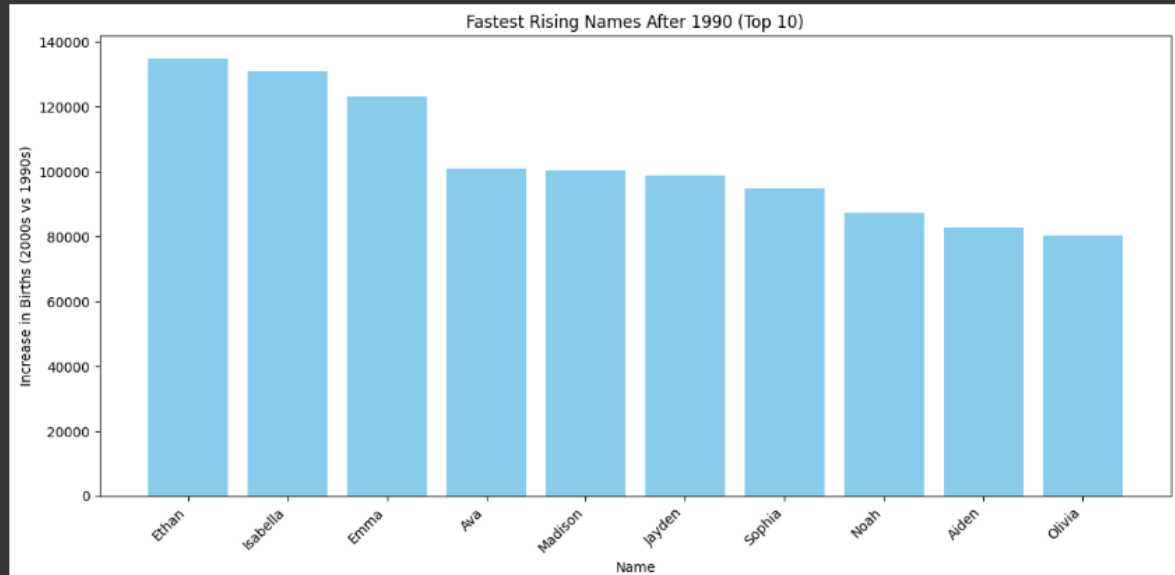
```
✓ 2s [1]
# Query 2: Fastest rising names after 1990
query_stephen_2 = """
WITH Name_90s AS (
  SELECT name, SUM(number) AS total_90s
  FROM `bigquery-public-data.usa_names.usa_1910_current`
  WHERE year BETWEEN 1990 AND 1999
  GROUP BY name
),
Name_2000s AS (
  SELECT name, SUM(number) AS total_2000s
  FROM `bigquery-public-data.usa_names.usa_1910_current`
  WHERE year BETWEEN 2000 AND 2009
  GROUP BY name
)
SELECT n2.name, n1.total_90s, n2.total_2000s, (n2.total_2000s - n1.total_90s) AS increase
FROM Name_90s n1
JOIN Name_2000s n2 ON n1.name = n2.name
ORDER BY increase DESC
LIMIT 10
"""

df_stephen_2 = run_query(query_stephen_2)
print("Stephen Query 2: Fastest Rising Names after 1990")
print(df_stephen_2)
```

	name	total_90s	total_2000s	increase
0	Ethan	66914	201931	135017
1	Isabella	18484	149591	131107
2	Emma	58213	181425	123212
3	Ava	3691	104534	100843
4	Madison	93465	193665	100200
5	Jayden	3506	102417	98911
6	Sophia	24296	119246	94950
7	Noah	56232	143677	87445
8	Aiden	1268	84046	82778
9	Olivia	75902	156104	80202

```
[14] import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 6))  
plt.bar(df_stephen_2['name'], df_stephen_2['increase'], color='skyblue')  
plt.xlabel('Name')  
plt.ylabel('Increase in Births (2000s vs 1990s)')  
plt.title('Fastest Rising Names After 1990 (Top 10)')  
plt.xticks(rotation=45, ha='right')  
plt.tight_layout()  
plt.show()
```



Andy's Queries

Query 1: Celebrity Influence — 'Kobe'

▼ Andy's Queries

✓
2s

[16]

```
# Query 1: Celebrity influence - 'Kobe'
query_andy_1 = """
SELECT year, SUM(number) AS total_births
FROM `bigquery-public-data.usa_names.usa_1910_current`
WHERE name = 'Kobe'
GROUP BY year
ORDER BY year
"""

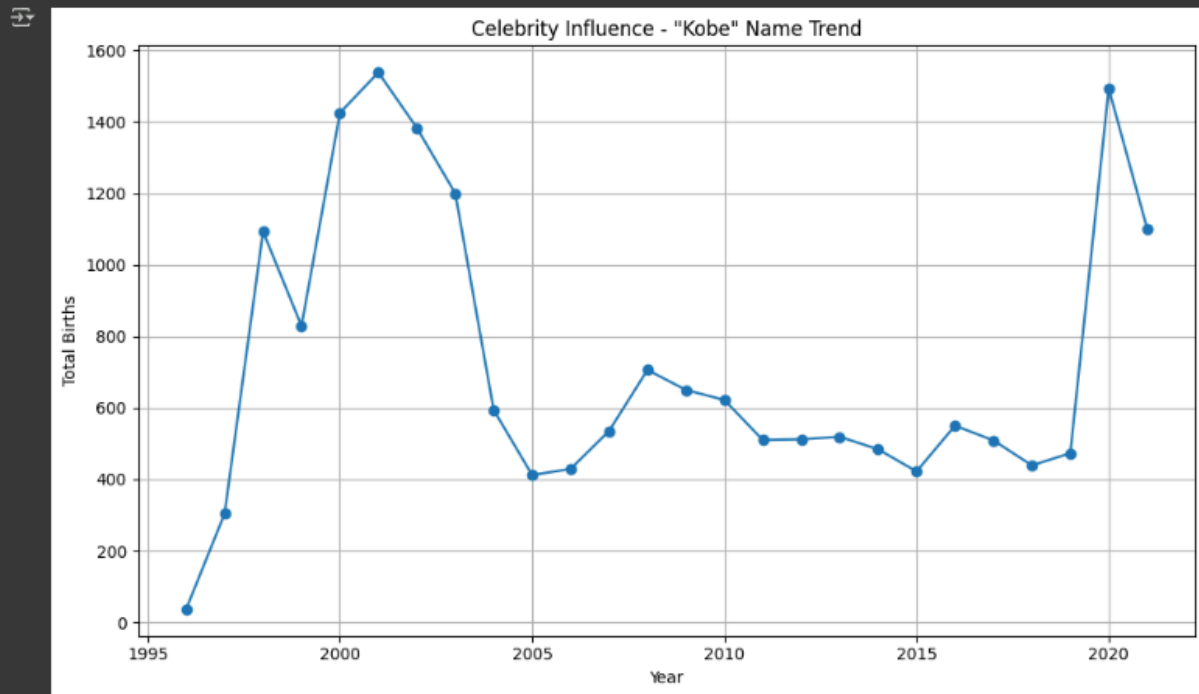
df_andy_1 = run_query(query_andy_1)
print("Andy Query 1: Celebrity Influence - 'Kobe' Name Trend")
print(df_andy_1)
```

➞ Andy Query 1: Celebrity Influence - 'Kobe' Name Trend

	year	total_births
0	1996	37
1	1997	304
2	1998	1093
3	1999	829
4	2000	1425
5	2001	1538
6	2002	1383
7	2003	1200
8	2004	594
9	2005	412
10	2006	429
11	2007	535
12	2008	706
13	2009	650
14	2010	622
15	2011	510
16	2012	512
17	2013	519
18	2014	484
19	2015	422
20	2016	550
21	2017	509
22	2018	439
23	2019	473
24	2020	1493
25	2021	1101

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(df_andy_1['year'], df_andy_1['total_births'], marker='o', linestyle='--')
plt.xlabel('Year')
plt.ylabel('Total Births')
plt.title('Celebrity Influence - "Kobe" Name Trend')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Query 2: Stable Names Across Decades

```
# Query 2: Stable names across decades
query_andy_2 = """
WITH Name_Year AS (
    SELECT name, year, SUM(number) AS total_births
    FROM `bigquery-public-data.usa_names.usa_1910_current`
    GROUP BY name, year
),

Name_Decade AS (
    SELECT
        name,
        CAST(year / 10 AS INT64) * 10 AS decade,
        SUM(total_births) AS total_births
    FROM Name_Year
    GROUP BY name, decade
),

Name_Decade_Stats AS (
    SELECT
        name,
        AVG(total_births) AS avg_births,
        STDDEV(total_births) AS stddev_births,
        COUNT(*) AS decades_count
    FROM Name_Decade
    GROUP BY name
    HAVING avg_births > 100 AND decades_count >= 2
)

SELECT
    name,
    avg_births AS overall_avg_births,
    stddev_births AS avg_stddev_births
FROM Name_Decade_Stats
ORDER BY avg_stddev_births ASC
LIMIT 10

"""

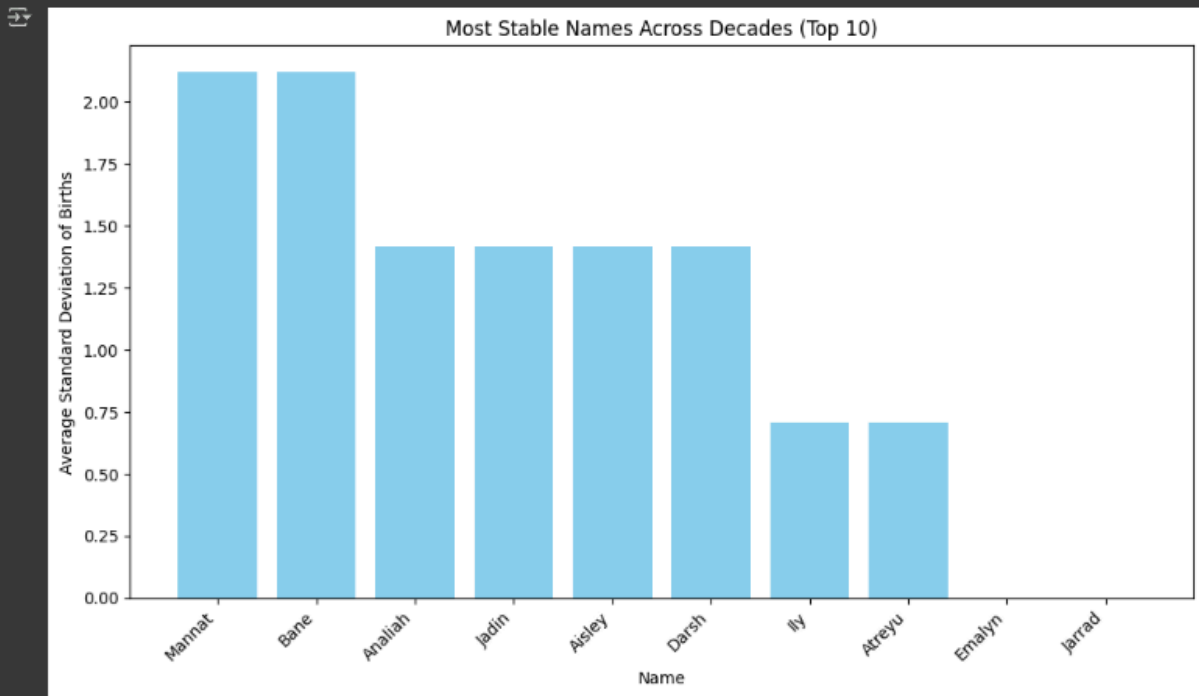
df_andy_2 = run_query(query_andy_2)
print("Andy Query 2: Most Stable Names Across Decades")
print(df_andy_2)
```

	name	overall_avg_births	avg_stddev_births
0	Jarrad	235.0	0.000000
1	Emalyn	347.0	0.000000
2	Ily	132.5	0.707107
3	Atreyu	513.5	0.707107
4	Darsh	144.0	1.414214
5	Aisley	105.0	1.414214
6	Jadin	347.0	1.414214
7	Analiah	191.0	1.414214
8	Bane	103.5	2.121320
9	Mannat	114.5	2.121320

```
import matplotlib.pyplot as plt

# Sort df_andy_2 by avg_stddev_births in descending order
df_andy_2_sorted = df_andy_2.sort_values(by=['avg_stddev_births'], ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(df_andy_2_sorted['name'], df_andy_2_sorted['avg_stddev_births'], color='skyblue')
plt.xlabel('Name')
plt.ylabel('Average Standard Deviation of Births')
plt.title('Most Stable Names Across Decades (Top 10)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Impact of optimization

As I already explained, the small query changes like narrowing year ranges and grouping data earlier made a huge difference in execution time and cost. This is a huge benefit when we work with large cloud datasets because it saves us time and money to run the query.

Suggestions for future improvements

For future improvement, we can incorporate clustering on a year or decade to increase speed time-based queries. We can also implement a rest api layer to expose insights for external applications.

Conclusions and Lessons Learned

This project is a huge opportunity for us to explore and run a large dataset on google cloud. We also learn how to make a small change in order to have huge improvement on reducing the IO cost. More importantly, we also learn from the data that showed how naming patterns reflect social and cultural evolution.